Lab 3 Architecture Justification

Airport Lost Luggage Claim and Tracking Portal

Mohammed Zahoor Mashahir | PES1UG24CS579 | Section J
Problem Statement 30

Architecture Selection

We chose Layered Architecture for the Airport Lost Luggage Claim & Tracking Portal System. The design separates presentation, business logic, and data access and integration. Business modules run within one application, while adapters isolate airport scan feeds and the compensation provider.

Reason 1 Consistent Claim and Compensation Records

A claim, its recovery updates, approval and payment record share a relational repository. Database transactions keep local state changes and their audit entries consistent. A unique active-claim constraint prevents duplicates; a durable payout outbox and an idempotency key support safe retries after approval. External payments are reconciled separately because they cannot share the local database transaction.

Reason 2 Replaceable Airport Integrations

The Baggage Feed Adapter normalizes scan formats from connected airports behind IBaggageFeed. Tracking can match a bag tag and itinerary against a common scan history without coupling claim forms or compensation rules to each airport API. Changes to an airport feed remain within its adapter.

Security Advantage

The Claim Workflow API validates authenticated sessions, claim ownership and agent authorization before routing requests. Passengers can access only their claims; agent updates and payout approvals require the assigned role and are audited. Clients have no direct database or payment-provider access. HTTPS protects external calls, and service credentials remain on the server.

Performance Benefit

In-process business calls avoid network hops between internal modules. Tracking reads an indexed local scan history instead of contacting every airport during a timeline request. This supports the NFR-001 target of p95 response time at most 200 ms, subject to load testing at 500 concurrent users. Background ingestion keeps scan retrieval off the interactive path.

Tradeoff

Individual modules cannot scale independently as microservices can. Stateless application replicas and database failover are deployment measures proposed for the 99.9% availability target; the layered pattern alone does not guarantee it.